

Doc. no. P0177R0
Date: 2016-02-15
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Cleaning up `allocator_traits`

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
2. [Introduction](#)
3. [Problems with `allocator\_traits`](#)
 - [Consistency of propagation traits](#)
4. [Acknowledgements](#)
5. [References](#)

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Introduction

The C++11 standard introduced `allocator_traits` with great flexibility as a way to generically customize allocator behavior. With experience, we have learned that some of this generality comes at great cost, but little benefit. This paper proposed cleaning up the wilder corners of the traits behavior.

Problems with `allocator_traits`

The addition of `allocator_traits` to C++11 brought relatively simple support for a wide variety of allocator models to the C++ standard library. It provides a consistent interface for containers and other types that require memory allocation services to be customized non-intrusively by their users. This traits template is deliberately designed to be flexible to support a wide variety of allocator models, as described in paper [P0176R0](#). However, not all of that generality is useful, and some creates needless complexity when implementing types that use allocators.

Consistency of propagation traits

One of the important customizations for stateful allocators is the ability to control the *propagation* of allocators, that is, to control which allocator is used after an assignment operator, or a `swap` operation. When all allocators of the same type are interchangeable, and so always compare equal, these traits have little value. If all allocator objects are interchangeable, then the only effect of propagation is potentially making a few more assignment or swaps. However, when allocators have state that affects allocation behavior, this can become critically important.

The first model, that drives the current default behavior that allocators do not propagate, is that all subcomponents of a data structure should use the same allocator. For example, a container would pass its own allocator down to its elements, and they in turn would pass that allocator down to their bases and members. Once this invariant is established, we do not want to lose it by swapping with elements for elsewhere that use a different allocator, or change allocator when assigned-to from an external source. This allows us to reason about the lifetime of the allocator and the data structure, see [P0176R0](#) for further details.

A second model is that the allocator move with the allocated memory, so every move-assignment and `swap` should propagate the allocator to maintain a non-throwing wide contract, and not risk an allocation or undefined behavior when the allocators do not match, but without any guarantee that a given data structure will have a consistent allocation strategy using such allocators.

There is no clear model that benefits from requiring container implementations to support allocators that propagate on move assignment, but not on `swap` (or vice-versa). The cost in terms of complexity this burdens container implementers with is considerable, and every operation involving another object that uses the same allocator family must document whether any potentially moving operation explicitly in terms whether it uses move-assignment, `swap`, or both. Use of an undocumented operation would result in a surprising allocator propagation which is distinctly observable.

The issue with copy-assignment is a little more subtle, but for a type that provides a copy-assignment operator but no move-assignment operator, then, when move-assignment is requested, the copy-assignment operator will be called. If the propagation traits of the user-supplied allocator differ, then problems will follow.

The proposed solution of this paper is to require that all three propagation traits be equivalent, and change the type-computation in the trait to simply delegate to a preferred trait. Customizing that one trait would change the behavior of all three. Note that this is a breaking change if anyone has successfully made use of diverging these traits, although the author has no experience of such allocators outside of standard conformance suites.

Once we mandate consistency, we could go further and deprecate the two names that are simply aliases of the first. Similarly, we would update all standard wording that references the two deprecated traits to use the one remaining propagation trait instead. Ideally, we would have a new trait

with a simpler name, but that would break all existing code where users have correctly customized the existing traits (consistently). Therefore, as we must pick one of the three existing names, we will pick the one that is shorter to type, as these names are infamously long when used (typically seen only by users implementing their own custom containers).

Inconsistent propagation traits are hard to deal with

This is the other side to 2.1, highlighting the burden on library implementers to handle inconsistent propagation traits. First, as we have three independant binary traits, that is 2^3 combinations of different behavior that must be tested, just for the allocator parameter of any container. These traits can have a subtle impact on many operations. As an example, consider the copy-assignment operator for `vector`. This has the strong exception safety guarantee, so we would like to use the copy and `swap` idiom, but how does that work if the propagation traits are different for copy-assignment and `swap`? Here is a sample implementation, relying on compiler optimizations to eliminate the dead branches (as the `if` expressions all yield a compile-time result).

```
template <typename T, typename A>
auto vector<T, A>::operator=(vector const& other) -> vector&
{
    using Traits = allocator_traits<A>

    if (!AT::propagate_on_copy_assignment) {
        vector temp(other, this->get_allocator());
        swap(temp);
    }
    else if (AT::propagate_on_swap) {
        vector temp(other, other.get_allocator());
        swap(temp);
    }
    else if ( AT::propagate_on_copy_assignment && !AT::propagate_on_swap) {
        // This is the really awkward case
        vector temp(::std::move(*this));
        try {
            this->~vector();
            new(this) vector(other, other.get_allocator());
        }
        catch(...) {
            new(this) vector(::std::move(temp));
            throw;
        }
    }

    return *this;
}
```

As an alternative to the destroy/in-place new idiom, the moved-from `vector` could be `cleared`, then use a private member function to rebind the allocator, followed by a range-insertion, and reverse the operation in the `catch` clause (as moving back after rebinding the allocator is a simple pointer operation, and cannot throw).

Note the return of an explicit `try/catch` block, or its moral equivalent by introducing some other guard class or scopeguard. This is exactly the kind of code construct that the copy/swap idiom is supposed to save us from, preferring to `throw` freely but `catch` rarely.

With the changes proposed in this paper, the example becomes a lot simpler (and there are fewer configurations to validate in a test driver):

```
template <typename T, typename A>
auto vector<T, A>::operator=(vector const& other) -> vector&
{
    using Traits = allocator_traits<A>
    Allocator alloc = AT::propagate
        ? other.get_allocator()
        : this->get_allocator();

    vector temp(other, alloc);
    swap(temp);

    return *this;
}
```

Default for propagation traits

Note:

- [LWG #2103](#) `std::allocator_traits<std::allocator<T>>::propagate_on_container_move_assignment`
- [LWG #2108](#) No way to identify allocator types that always compare equal
- [N4258](#) Cleaning up `noexcept` in the Library, Nicolai Josuttis

noexcept Move Operations

Two of the key operations of a container are its move-assignment operator and its `swap` overload. While it is important that these are efficient, it is also important that callers can observe whether they have the no-throw guarantee, or if they need to code defensively around these operations. Nicolai Jossutis made an important contribution earlier in the C++17 process, introducing the `is_always_equal` predicate to identify the case where allocators that do not propagate can still give the no-throw guarantee for their container's operations. Unfortunately, this makes the signature of these key operations appear much more complex than necessary. This is also very visible in the specification, as these are the important operations that will be advertized frequently. Worse, we expect implementers of their own containers to duplicate this logic and spelling in their own implementations of these functions.

This paper proposes adding a new `constexpr` variable template with a simpler spelling in the `std` namespace, that should also be clearer to read in the specification of the critical operations. Finding a good name, that is simple, short, and precise, is difficult. The initially suggest name is `propagate_may_throw_v`, although it is expected that the Library Evolution Working Group will brainstorm something better.

In addition to `basic_string` and `vector`, there are several other containers that provide a no-throw guarantee when the allocators guarantee to compare equal, should we use this new `propagate_may_throw_v` exception-specification there as well? While the consistency is appealing, it would also break some existing implementations, as mentioned in Nico's paper.

The key to the issue is that, even if allocators can propagate on move-assignment, we expect them to propagate in only one direction. For containers that need a sentry node to maintain their invariants, a new sentry must be allocated with the moved allocator, which might throw. This should not be an issue for `swap` operations though, as allocators are expected to be exchanged, along with two data structures that satisfy the invariants. A second issue arises for containers that hold predicates or other function objects in addition to the stored elements, as even though a swap may not need to allocate, the functor objects may throw on move/swap. This limits our scope to make changes, although it looks like `deque::swap` should take advantage.

Tidying the Allocator Requirements Table

The allocator requirements table in clause 17 is long, so long that it spreads over three pages of the standard. It is also complex with a lot of small identifiers used to simplify the specification, and there are so many of these that they have their own table in advance of the allocator requirements table that is the key to that table. Unfortunately, the spelling of the short identifiers is not always obvious, which necessitates scrolling up and down a few pages, every time the reader wants to clarify their understanding. Worse, in some cases it is outright misleading, where `x` is the type of an allocator, but `x` is an object of a pointer type, not an allocator. Instead, and allocator of type `x` is spelled `a`. This paper proposes a more intuitive set of spellings that are similarly terse, but will give the reader an intuition they can trust without scrolling back to the table key each time.

Secondly, around half of the entries in the allocator requirements table are optional, denoted by the presence of the Defaults column. However, most operations are specified directly in terms of the allocator member, when really they should be specified in terms of the default, which is obtained through `allocator_traits`. There would be a risk of a circular definition if the specification of `allocator_traits` were in terms of this table, but that is not the case. The specification for `allocator_traits` gives a formula for the default in each case without (normatively) referring back to the allocator requirements table. This paper substitutes the `allocator_traits` name for every optional property that is used in the specification of dependent requirements.

Ideas not Pursued

There were a few additional ideas that occurred during the writing of this paper that the author ultimately rejected.

Introduce a new name for the propagation trait

Retaining a single propagation trait with the term `swap` as part of its name is not ideal, as it suggests the original, more specific meaning of the trait. This could be resolved by having a single `propagate_allocator` trait, possible as a free-standing variable template, `propagate_allocator_v`. Its default value could be computed in some way from the existing three (deprecated) legacy traits, in an attempt to provide some backwards-compatibility scheme for existing allocators. Alternatively, the three deprecated traits in `allocator_traits` could simply check this value, supporting existing container implementations (which are likely to be more numerous).

Ultimately this idea was rejected as being too big a change, providing a much more awkward period of transition. Generally, a feature is not simplified by adding more redundant names for the existing behavior.

However, a new `propagate_may_throw_v` trait is proposed, that simplifies the frequent combination of checking for an allocator that either propagates, or for which comparison is `always true`. This case is encountered frequently enough in the standard that it should be simplified.

Drive-by Fixes

The Library Active Issues Lists was consulted for any allocator related issues that might be resolved as part of this clean-up exercise. In addition to issues directly addressing allocators, a few issues addressing class definitions that would be partially addressed by fixing allocator support were, instead, fully addressed to avoid confusion with multiple resolutions interacting on the same wording.

Annex C was not updated for C++11

- [LWG #2178](#) Annex C update for C++11 on allocators

Regex `match_results` does not use `noexcept`

- [LWG #2183](#) `regex::match_results` missing allocator-aware ctors
- [LWG #2184](#) `regex::match_results` assignment and allocator propagation

Regex `match_results` does not use `noexcept`

- [LWG #2490](#) `<regex>` needs lots of `noexcept` Note that this issue is not fully addressed by this paper, but only one class.

`vector<bool>` does not use `noexcept`

Adding (nothrow) swappable traits

This paper should incorporate the improved support for portable detection of `noexcept` swap functions in [N4511](#). Adding [nothrow]-swappable traits,

Proposed Wording

Amend existing library clauses as below:

17.6.3.5 Allocator requirements [allocator.requirements]

1. The library describes a standard set of requirements for allocators, which are class-type objects that encapsulate the information about an allocation model. This information includes the knowledge of pointer types, the type of their difference, the type of the size of objects in this allocation model, as well as the memory allocation and deallocation primitives for it. All of the string types (Clause 21), containers (Clause 23) (except array), string buffers and string streams (Clause 27), and match_results (Clause 28) are parameterized in terms of allocators.
2. The class template allocator_traits (20.7.8) supplies a uniform interface to all allocator types. Table 27 describes the types manipulated through allocators. Table 28 describes the requirements on allocator types and thus on types used to instantiate allocator_traits. A requirement is optional if the last column of Table 28 specifies a default for a given expression. Within the standard library allocator_traits template, an optional requirement that is not supplied by an allocator is replaced by the specified default expression. A user specialization of allocator_traits may provide different defaults and may provide defaults for different requirements than the primary template. Within Tables 27 and 28, the use of move and forward always refers to std::move and std::forward, respectively.

Table 27 “Descriptive variable definitions

Variable	Definition
T, U, C	any cv-unqualified object type (3.9)
$\mathcal{A}$	an Allocator class for type τ
$\mathcal{B}$	the corresponding Allocator class for type υ
$\mathcal{A}\mathcal{T}$	the type allocator_traits< $\mathcal{A}$ >
$\mathcal{B}\mathcal{T}$	the type allocator_traits< $\mathcal{B}$ >
a, a1, a2	lvalues of type $\mathcal{A}\mathcal{X}$
u	the name of a variable being declared
b	a value of type $\mathcal{B}\mathcal{Y}$
c*	a pointer of type C* through which indirection is valid
p	a value of type $\mathcal{A}\mathcal{T}::\text{pointer}$, obtained by calling a1.allocate, where a1 == a
qcp	a value of type $\mathcal{A}\mathcal{T}::\text{const_pointer}$ obtained by conversion from a value p.
wvp	a value of type $\mathcal{A}\mathcal{T}::\text{void_pointer}$ obtained by conversion from a value p
xcvp	a value of type $\mathcal{A}\mathcal{T}::\text{const_void_pointer}$ obtained by conversion from a value qcp or a value wvp
ycvq	a value of type $\mathcal{A}\mathcal{T}::\text{const_void_pointer}$ obtained by conversion from a result value of $\mathcal{B}\mathcal{T}::\text{allocate}$, or else a value of type (possibly const) std::nullptr_t.
n	a value of type $\mathcal{A}\mathcal{T}::\text{size_type}$.
Args	a template parameter pack
args	a function parameter pack with the pattern Args&&

Table 28 “Allocator requirements

Expression	Return type	Assertion/note pre-/post-condition	Default
$\mathcal{A}::\text{pointer}$			T*
$\mathcal{A}::\text{const_pointer}$		$\mathcal{A}\mathcal{T}::\text{pointer}$ is convertible to $\mathcal{A}\mathcal{T}::\text{const_pointer}$	pointer_traits< $\mathcal{A}\mathcal{T}::\text{pointer}$ >::rebind<const T>
$\mathcal{A}::\text{void_pointer}$ $\mathcal{B}::\text{void_pointer}$		$\mathcal{A}\mathcal{T}::\text{pointer}$ is convertible to $\mathcal{A}\mathcal{T}::\text{void_pointer}$. $\mathcal{A}\mathcal{T}::\text{void_pointer}$ and $\mathcal{B}\mathcal{T}::\text{void_pointer}$ are the same type.	pointer_traits< $\mathcal{A}\mathcal{T}::\text{pointer}$ >::rebind<void>
$\mathcal{A}::\text{const_void_pointer}$ $\mathcal{B}::\text{const_void_pointer}$		$\mathcal{A}\mathcal{T}::\text{pointer}$, $\mathcal{A}\mathcal{T}::\text{const_pointer}$, and $\mathcal{A}\mathcal{T}::\text{void_pointer}$ are convertible to $\mathcal{A}\mathcal{T}::\text{const_void_pointer}$. $\mathcal{A}\mathcal{T}::\text{const_void_pointer}$ and $\mathcal{B}\mathcal{T}::\text{const_void_pointer}$ are the same type.	pointer_traits< $\mathcal{A}\mathcal{T}::\text{pointer}$ >::rebind<const void>
$\mathcal{A}::\text{value_type}$	Identical to τ		
$\mathcal{A}::\text{size_type}$	unsigned integer type	a type that can represent the size of the largest object in the allocation model.	
$\mathcal{A}::\text{difference_type}$	signed integer type	a type that can represent the difference between any two pointers in the allocation	

		model.	
typename $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{template rebind}<\text{U}>::\text{other}$	$\text{\textcolor{red}{Y}}\text{\textcolor{green}{B}}$	For all u (including τ), $\text{\textcolor{red}{Y}}\text{\textcolor{green}{B}}::\text{template rebind}<\text{T}>::\text{other}$ is $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}$	See Note A, below.
*p	T&		
* $\text{\textcolor{red}{q}}\text{\textcolor{green}{cp}}$	const T&	* $\text{\textcolor{red}{q}}\text{\textcolor{green}{cp}}$ refers to the same object as *p	
p->m	type of $\text{T}::\text{m}$	<i>pre:</i> (*p).m is well-defined. equivalent to (*p).m	
$\text{\textcolor{red}{q}}\text{\textcolor{green}{cp}}->\text{m}$	type of $\text{T}::\text{m}$	<i>pre:</i> (* $\text{\textcolor{red}{q}}\text{\textcolor{green}{cp}}$).m is well-defined. equivalent to (* $\text{\textcolor{red}{q}}\text{\textcolor{green}{cp}}$).m	
static_cast< $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{pointer}>(\text{\textcolor{red}{w}}\text{\textcolor{green}{vp}})$	pointer	static_cast< $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{pointer}>(\text{\textcolor{red}{w}}\text{\textcolor{green}{vp}}) == p$	
static_cast< $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{const_pointer}>(\text{\textcolor{red}{x}}\text{\textcolor{green}{cvp}})$	const_pointer	static_cast< $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{const_pointer}>(\text{\textcolor{red}{x}}\text{\textcolor{green}{cvp}}) == p$	
a.allocate(n)	pointer	Memory is allocated for n objects of type τ but objects are not constructed. allocate may raise an appropriate exception. ¹⁸⁰ [<i>Note:</i> If $n == 0$, the return value is unspecified. <i>â€” end note</i>]	
a.allocate(n, $\text{\textcolor{red}{y}}\text{\textcolor{green}{cvq}}$)	pointer	Same as a.allocate(n). The use of $\text{\textcolor{red}{y}}\text{\textcolor{green}{cvq}}$ is unspecified, but it is intended as an aid to locality.	a.allocate(n)
a.deallocate(p,n)	(not used)	<i>pre:</i> p shall be a value returned by an earlier call to allocate that has not been invalidated by an intervening call to deallocate. n shall match the value passed to allocate to obtain this memory. <i>Throws:</i> Nothing.	
a.max_size()	$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{size_type}$	the largest value that can meaningfully be passed to $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}::\text{allocate}()$	numeric_limits<size_type>::max()/sizeof(value_type)
a1 == a2	bool	returns true only if storage allocated from each can be deallocated via the other. operator== shall be reflexive, symmetric, and transitive, and shall not exit via an exception.	
a1 != a2	bool	same as !(a1 == a2)	
a == b	bool	same as a == $\text{\textcolor{red}{Y}}\text{\textcolor{green}{B}}::\text{rebind}<\text{T}>::\text{other}(b)$	
a != b	bool	same as !(a == b)	
$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u}(a)$; $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u} = a$;		Shall not exit via an exception. post: a1 == a	
$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u}(b)$;		Shall not exit via an exception. post: $\text{\textcolor{red}{Y}}\text{\textcolor{green}{B}}(a) == b$, a == $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}(b)$	
$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u}(\text{move}(a))$; $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u} = \text{move}(a)$;		Shall not exit via an exception. post: a1 equals the prior value of a.	
$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}\text{ u}(\text{move}(b))$;		Shall not exit via an exception. post: a equals the prior value of $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}(b)$.	
a.construct(c,args)	(not used)	Effect: Constructs an object of type c at c	::new ((void*)c) C(forward<Args>(args)...)
a.destroy(c)	(not used)	Effect: Destroys the object at c	c->~C()
a.select_on_container_copy_construction()	$\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}$	Typically returns either a or $\text{\textcolor{red}{X}}\text{\textcolor{green}{A}}()$	return a;
propagate_on_container_copy_assignment	Identical to or derived from true_type or false_type	true_type only if an allocator of type x should be copied when the client container is copy-assigned. See Note B, below.	false_type
propagate_on_container_move_assignment	Identical to or derived from true_type or	true_type only if an allocator of type x should be moved when the client container is move-assigned. See Note B,	false_type

	false_type	below.	
propagate_on_container_swap	Identical to or derived from true_type or false_type	true_type only if an allocator of type $\mathbb{A}$ should be swapped when the client container is swapped, copied when the client container is copy-assigned, and moved when the client container is move-assigned. See Note B, below.	false_type
is_always_equal	Identical to or derived from true_type or false_type	true_type only if the expression $a1 == a2$ is guaranteed to be true for any two (possibly const) values $a1$, $a2$ of type $\mathbb{A}$.	is_empty $\mathbb{T} < \mathbb{A} > :: \text{type}$

- Note A: The member class template `rebind` in the table above is effectively a typedef template. [*Note:* In general, if the name `Allocator` is bound to `SomeAllocator<T>`, then `Allocator::rebind<U>::other` is the same type as `SomeAllocator<U>`, where `SomeAllocator<T>::value_type` is `T` and `SomeAllocator<U>::value_type` is `U`. *end note*] If `Allocator` is a class template instantiation of the form `SomeAllocator<T, Args>`, where `Args` is zero or more type arguments, and `Allocator` does not supply a `rebind` member template, the standard `allocator_traits` template uses `SomeAllocator<U, Args>` in place of `Allocator::rebind<U>::other` by default. For allocator types that are not template instantiations of the above form, no default is provided.
- Note B: If `AT::propagate_on_container_swap::value` is true, $\mathbb{A}$ shall satisfy the `CopyAssignable` requirements (Table 23) and the copy operation shall not throw exceptions. If `AT::propagate_on_container_swapmove::value` is true, $\mathbb{A}$ shall satisfy the `MoveAssignable` requirements (Table 22) and the move operation shall not throw exceptions. If `AT::propagate_on_container_swap::value` is true, lvalues of type $\mathbb{A}$ shall be swappable (17.6.3.2) and the swap operation shall not throw exceptions.
- An allocator type $\mathbb{A}$ shall satisfy the requirements of `CopyConstructible` (17.6.3.1). The `AT::pointer`, `AT::const_pointer`, `AT::void_pointer`, and `AT::const_void_pointer` types shall satisfy the requirements of `NullablePointer` (17.6.3.3). No constructor, comparison operator, copy operation, move operation, or swap operation on these pointer types shall exit via an exception. `AT::pointer` and `AT::const_pointer` shall also satisfy the requirements for a random access iterator (24.2).
- Let $x1$ and $x2$ denote objects of (possibly different) types `AT::void_pointer`, `AT::const_void_pointer`, `AT::pointer`, or `AT::const_pointer`. Then, $x1$ and $x2$ are equivalently-valued pointer values, if and only if both $x1$ and $x2$ can be explicitly converted to the two corresponding objects $px1$ and $px2$ of type `AT::const_pointer`, using a sequence of `static_casts` using only these four types, and the expression $px1 == px2$ evaluates to true.
- Let $w1$ and $w2$ denote objects of type `AT::void_pointer`. Then for the expressions

```
w1 == w2
w1 != w2
```

either or both objects may be replaced by an equivalently-valued object of type `AT::const_void_pointer` with no change in semantics.

- Let $p1$ and $p2$ denote objects of type `AT::pointer`. Then for the expressions

```
p1 == p2
p1 != p2
p1 < p2
p1 <= p2
p1 >= p2
p1 > p2
p1 - p2
```

either or both objects may be replaced by an equivalently-valued object of type `AT::const_pointer` with no change in semantics.

- An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct` or `destroy` members may be called. If a type cannot be used with a particular allocator, the allocator class or the call to `construct` or `destroy` may fail to instantiate. [Example: the following is an allocator class template supporting the minimal interface that satisfies the requirements of Table 28:

```
template <class Tp>
struct SimpleAllocator {
    typedef Tp value_type;
    SimpleAllocator(ctor args);

    template <class T> SimpleAllocator(const SimpleAllocator<T>& other);

    Tp* allocate(std::size_t n);
    void deallocate(Tp* p, std::size_t n);
};

template <class T, class U>
bool operator==(const SimpleAllocator<T>&, const SimpleAllocator<U>&);
template <class T, class U>
bool operator!=(const SimpleAllocator<T>&, const SimpleAllocator<U>&);
```

end example]

- If the alignment associated with a specific over-aligned type is not supported by an allocator, instantiation of the allocator for that type may fail. The allocator also may silently ignore the requested alignment. [Note: Additionally, the member function `allocate` for that type may fail by throwing an object of type `std::bad_alloc`. *end note*]

17.6.3.5.1 Allocator completeness requirements [allocator.requirements.completeness]

1. If `xA` is an allocator class for type `T`, `xA` additionally satisfies the allocator completeness requirements if, whether or not `T` is a complete type:
 1. `xA` is a complete type, and
 2. all the member types of `allocator_traits<xA>` 20.7.8 other than `value_type` are complete types

20.7.8 Allocator traits [allocator.traits]

1. The class template `allocator_traits` supplies a uniform interface to all allocator types. An allocator cannot be a non-class type, however, even if `allocator_traits` supplies the entire required interface. [*Note:* Thus, it is always possible to create a derived class from an allocator. - *end note*]

```
namespace std {
    template <class Alloc> struct allocator_traits {
        typedef Alloc allocator_type;

        typedef typename Alloc::value_type value_type;

        typedef see below pointer;
        typedef see below const_pointer;
        typedef see below void_pointer;
        typedef see below const_void_pointer;

        typedef see below difference_type;
        typedef see below size_type;

        typedef see below propagate_on_container_copy_assignment;
        typedef see below propagate_on_container_move_assignment;
        typedef see below propagate_on_container_swap;
        typedef see below is_always_equal;

        template <class T> using rebind_alloc = see below;
        template <class T> using rebind_traits = allocator_traits<rebind_alloc<T> >;

        static pointer allocate(Alloc& a, size_type n);
        static pointer allocate(Alloc& a, size_type n, const_void_pointer hint);

        static void deallocate(Alloc& a, pointer p, size_type n);

        template <class T, class... Args>
            static void construct(Alloc& a, T* p, Args&&... args);

        template <class T>
            static void destroy(Alloc& a, T* p);

        static size_type max_size(const Alloc& a) noexcept;

        static Alloc select_on_container_copy_construction(const Alloc& rhs);
    };
}

// NOTE: MOVE THIS TO SYNOPSIS INSTEAD
template <class Alloc>
constexpr propagate_may_throw v =
    !allocator_traits<Alloc>::is_always_equal
    !allocator_traits<Alloc>::propagate_on_container_swap;
```

20.7.8.1 Allocator traits member types [allocator.traits.types]

```
typedef see below propagate_on_container_copy_assignment;
```

7. *Type:* `Alloc::propagate_on_container_copy_assignment` if the *qualified-id* `Alloc::propagate_on_container_copy_assignment` is valid and denotes a type (14.8.2); otherwise `false_type` `propagate_on_container_move_assignment`.

```
typedef see below propagate_on_container_move_assignment;
```

8. *Type:* `Alloc::propagate_on_container_move_assignment` if the *qualified-id* `Alloc::propagate_on_container_move_assignment` is valid and denotes a type (14.8.2); otherwise `false_type` `is_always_equal`.

```
typedef see below propagate_on_container_swap;
```

9. *Type:* `Alloc::propagate_on_container_swap` if the *qualified-id* `Alloc::propagate_on_container_swap` is valid and denotes a type (14.8.2); otherwise `false_type` `is_always_equal`.

```
typedef see below is_always_equal;
```

10. *Type:* `Alloc::is_always_equal` if the *qualified-id* `Alloc::is_always_equal` is valid and denotes a type (14.8.2); otherwise `is_empty<Alloc>::type`.

20.13.1 Header <scoped_allocator> synopsis

```
namespace std {
    template <class OuterAlloc, class... InnerAllocs>
        class scoped_allocator_adaptor : public OuterAlloc {

        typedef see below propagate_on_container_copy_assignment;
        typedef see below propagate_on_container_move_assignment;
        typedef see below propagate_on_container_swap;
```

```

        typedef see below is_always_equal;
    };
}

```

20.13.2 Scoped allocator adaptor member types [allocator.adaptor.types]

```

typedef see below inner_allocator_type;

```

1. **Type:** `scoped_allocator_adaptor<OuterAlloc>` if `sizeof... (InnerAllocs)` is zero; otherwise, `scoped_allocator_adaptor<InnerAllocs...>`.

```

typedef see below propagate_on_container_copy_assignment;

```

2. **Type:** `true_type` if `allocator_traits<A>::propagate_on_container_copy_assignment::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

```

typedef see below propagate_on_container_move_assignment;

```

3. **Type:** `true_type` if `allocator_traits<A>::propagate_on_container_move_assignment::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

```

typedef see below propagate_on_container_swap;

```

4. **Type:** `true_type` if `allocator_traits<A>::propagate_on_container_swap::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

```

typedef see below is_always_equal;

```

5. **Type:** `true_type` if `allocator_traits<A>::is_always_equal::value` is true for every A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

21.4 Class template `basic_string` [basic.string]

```

namespace std {
    template <class charT, class traits = char_traits<charT>,
              class Allocator = allocator<charT> >
    class basic_string {

    ~basic_string();
    basic_string& operator=(const basic_string& str);
    basic_string& operator=(basic_string&& str)
        noexcept(!propagate_may_throw_v<Allocator>)
        noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
                  allocator_traits<Allocator>::is_always_equal::value);
    basic_string& operator=(const charT* s);
    basic_string& operator=(charT c);
    basic_string& operator=(initializer_list<charT>);

    basic_string& assign(const basic_string& str);
    basic_string& assign(basic_string&& str)
        noexcept(!propagate_may_throw_v<Allocator>)
        noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
                  allocator_traits<Allocator>::is_always_equal::value);
    basic_string& assign(const basic_string& str, size_type pos,
                        size_type n = npos);
    basic_string& assign(const charT* s, size_type n);
    basic_string& assign(const charT* s);
    basic_string& assign(size_type n, charT c);
    template<class InputIterator>
        basic_string& assign(InputIterator first, InputIterator last);
    basic_string& assign(initializer_list<charT>);

    void swap(basic_string& str)
        noexcept(!propagate_may_throw_v<Allocator>)
        noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
                  allocator_traits<Allocator>::is_always_equal::value);
    };
}

```

23.2.1 General container requirements [container.requirements.general]

8 Unless otherwise specified, all containers defined in this clause obtain memory using an allocator (see 17.6.3.5). Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. Such move construction of the allocator shall not exit via an exception. All other constructors for these container types take a `const allocator_type&` argument. [Note: If an invocation of a constructor uses the default value of an optional allocator argument, then the `Allocator` type must support value initialization. - end note] A copy of this allocator is used for any memory allocation and element construction performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value`, `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value`, or `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true within the implementation of the corresponding container

operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement.

9 The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than `array`, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Lvalues of any `Compare`, `Pred`, or `Hash` types belonging to `a` and `b` shall be swappable and shall be exchanged by calling `swap` as described in 17.6.3.2. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 17.6.3.2. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.

Table 98 “ Allocator-aware container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>a = rv</code>	<code>X&</code>	<i>Requires:</i> If <code>allocator_traits<allocator_type>::propagate_on_container_move_assignment::value</code> is false <code>propagate_may_throw_v<allocator_type></code> , <code>T</code> is <code>MoveInsertable</code> into <code>X</code> and <code>MoveAssignable</code> . All existing elements of <code>a</code> are either move assigned to or destroyed. <i>post:</i> <code>a</code> shall be equal to the value that <code>rv</code> had before this assignment.	linear

23.3.3.1 Class template deque overview [deque.overview]

```
namespace std {
    template <class T, class Allocator = allocator<T> >
    class deque {
        // most content elided for simplicity

        void swap(deque&) noexcept(!propagate_may_throw_v<Allocator>);
        noexcept(allocator_traits<Allocator>::is_always_equal::value);
    };
}
```

23.3.4.1 Class template forward_list overview [forwardlist.overview]

```
namespace std {
    template <class T, class Allocator = allocator<T> >
    class forward_list {
        // most content elided for simplicity

        void swap(forward_list&) noexcept(!propagate_may_throw_v<Allocator>);
        noexcept(allocator_traits<Allocator>::is_always_equal::value);
    };
}
```

23.3.5.1 Class template forward_list overview [list.overview]

```
namespace std {
    template <class T, class Allocator = allocator<T> >
    class forward_list {
        // most content elided for simplicity

        void swap(list&) noexcept(!propagate_may_throw_v<Allocator>);
        noexcept(allocator_traits<Allocator>::is_always_equal::value);
    };
}
```

23.3.6.1 Class template vector overview [vector.overview]

```
namespace std {
    template <class T, class Allocator = allocator<T> >
    class vector {

        ~vector();
        vector& operator=(const vector& x);
        vector& operator=(vector&& x)
            noexcept(!propagate_may_throw_v<Allocator>)
            noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
                allocator_traits<Allocator>::is_always_equal::value);
        vector& operator=(initializer_list<T>);

        void swap(vector&)
            noexcept(!propagate_may_throw_v<Allocator>)
            noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
                allocator_traits<Allocator>::is_always_equal::value);
    };
}
```

23.3.7 Class vector<bool> [vector.bool]

1. To optimize space allocation, a specialization of `vector` for `bool` elements is provided:

```

namespace std {
    template <class Allocator> class vector<bool, Allocator> {
    public:

        // construct/copy/destroy:
        vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }
        explicit vector(const Allocator&) noexcept;
        explicit vector(size_type n, const Allocator& = Allocator());
        vector(size_type n, const bool& value,
            const Allocator& = Allocator());
        template <class InputIterator>
            vector(InputIterator first, InputIterator last,
                const Allocator& = Allocator());
        vector(const vector<bool, Allocator>& x);
        vector(vector<bool, Allocator>&& x) noexcept;
        vector(const vector&, const Allocator&);
        vector(vector&&, const Allocator&);
        vector(initializer_list<bool>, const Allocator& = Allocator());
        ~vector();
        vector<bool, Allocator>& operator=(const vector<bool, Allocator>& x);
        vector<bool, Allocator>& operator=(vector<bool, Allocator>&& x)
            noexcept(!propagate_may_throw_v<Allocator>);
        vector& operator=(initializer_list);
        template <class InputIterator>
            void assign(InputIterator first, InputIterator last);
        void assign(size_type n, const bool& t);
        void assign(initializer_list<bool>);
        allocator_type get_allocator() const noexcept;

        iterator erase(const_iterator position);
        iterator erase(const_iterator first, const_iterator last)
            void swap(vector<bool, Allocator>&)
                noexcept(!propagate_may_throw_v<Allocator>);
        static void swap(reference x, reference y) noexcept;
        void flip() noexcept; // flips all bits
        void clear() noexcept;
    };
}

```

23.4.4.1 Class template map overview [map.overview]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T> > >
    class map {
    public:
        void swap(map&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                noexcept(!propagate_may_throw_v<Allocator> &&
                    noexcept(swap(declval<Compare>(), declval<Compare>()))));
    };
}

```

23.4.5.1 Class template multimap overview [multimap.overview]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T> > >
    class multimap {
    public:
        void swap(multimap&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                noexcept(!propagate_may_throw_v<Allocator> &&
                    noexcept(swap(declval<Compare>(), declval<Compare>()))));
    };
}

```

23.4.6.1 Class template set overview [set.overview]

```

namespace std {
    template <class Key, class Compare = less<Key>,
        class Allocator = allocator<Key> >
    class set {
    public:
        void swap(set&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                noexcept(!propagate_may_throw_v<Allocator> &&
                    noexcept(swap(declval<Compare>(), declval<Compare>()))));
    };
}

```

23.4.7.1 Class template multiset overview [multiset.overview]

```

namespace std {
    template <class Key, class Compare = less<Key>,
        class Allocator = allocator<Key> >
    class multiset {
    public:
        void swap(multiset&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                noexcept(!propagate_may_throw_v<Allocator> &&

```

```

    noexcept(swap(declval<Compare&>(), declval<Compare&>()));
};
}

```

23.5.4.1 Class template unordered_map overview [unord.map.overview]

```

namespace std {
    template <class Key,
              class T,
              class Hash = hash<Key>,
              class Pred = std::equal_to<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class unordered_map {
    void swap(unordered_map&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                 noexcept(!propagate_may_throw_v<Allocator> &&
                          noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
                          noexcept(swap(declval<Pred&>(), declval<Pred&>()))));
    };
}

```

23.5.5.1 Class template unordered_multimap overview [unord.multimap.overview]

```

namespace std {
    template <class Key,
              class T,
              class Hash = hash<Key>,
              class Pred = std::equal_to<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class unordered_multimap {
    void swap(unordered_multimap&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                 noexcept(!propagate_may_throw_v<Allocator> &&
                          noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
                          noexcept(swap(declval<Pred&>(), declval<Pred&>()))));
    };
}

```

23.5.6.1 Class template unordered_set overview [unord.set.overview]

```

namespace std {
    template <class Key,
              class Hash = hash<Key>,
              class Pred = std::equal_to<Key>,
              class Allocator = allocator<const Key> >
    class unordered_set {
    void swap(unordered_set&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                 noexcept(!propagate_may_throw_v<Allocator> &&
                          noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
                          noexcept(swap(declval<Pred&>(), declval<Pred&>()))));
    };
}

```

23.5.6.1 Class template unordered_multiset overview [unord.multiset.overview]

```

namespace std {
    template <class Key,
              class Hash = hash<Key>,
              class Pred = std::equal_to<Key>,
              class Allocator = allocator<const Key> >
    class unordered_multiset {
    void swap(unordered_multiset&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
                 noexcept(!propagate_may_throw_v<Allocator> &&
                          noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
                          noexcept(swap(declval<Pred&>(), declval<Pred&>()))));
    };
}

```

28.10 Class template match_results [re.results]

```

namespace std {
    template <class BidirectionalIterator,
              class Allocator = allocator<sub_match<BidirectionalIterator>>>
    class match_results {
    public:
        typedef sub_match<BidirectionalIterator> value_type;
        typedef const value_type& const_reference;
        typedef value_type& reference;
        typedef implementation_defined const_iterator;
        typedef const_iterator iterator;
        typedef typename iterator_traits<BidirectionalIterator>::difference_type difference_type;
        typedef typename allocator_traits<Allocator>::size_type size_type;
        typedef Allocator allocator_type;
        typedef typename iterator_traits<BidirectionalIterator>::value_type char_type;
    };
}

```

```

typedef basic_string<char_type>                string_type;

// 28.10.1, construct/copy/destroy:
explicit match_results(const Allocator& a = Allocator());
match_results(const match_results& m);
match_results(match_results&& m) noexcept;
match_results(const match_results& m, const Allocator& a);
match_results(match_results&& m, const Allocator& a);
match_results& operator=(const match_results& m);
match_results& operator=(match_results&& m)
    noexcept(!propagate_may_throw_v<Allocator>);
~match_results();

// 28.10.2, state:
bool ready() const noexcept;

// 28.10.3, size:
size_type size() const noexcept;
size_type max_size() const noexcept;
bool empty() const noexcept;

// 28.10.4, element access:
difference_type length(size_type sub = 0) const;
difference_type position(size_type sub = 0) const;
string_type str(size_type sub = 0) const;
const_reference operator[](size_type n) const;

const_reference prefix() const;
const_reference suffix() const;
const_iterator begin() const noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// 28.10.5, format:
template <class OutputIter>
    OutputIter
        format(OutputIter out,
            const char_type* fmt_first, const char_type* fmt_last,
            regex_constants::match_flag_type flags =
                regex_constants::format_default) const;
template <class OutputIter, class ST, class SA>
    OutputIter
        format(OutputIter out,
            const basic_string<char_type, ST, SA>& fmt,
            regex_constants::match_flag_type flags =
                regex_constants::format_default) const;
template <class ST, class SA>
    basic_string<char_type, ST, SA>
        format(const basic_string<char_type, ST, SA>& fmt,
            regex_constants::match_flag_type flags =
                regex_constants::format_default) const;
string_type
    format(const char_type* fmt,
        regex_constants::match_flag_type flags =
            regex_constants::format_default) const;

// 28.10.6, allocator:
allocator_type get_allocator() const noexcept;

// 28.10.7, swap:
void swap(match_results& that)
    noexcept(!propagate_may_throw_v<Allocator>);
};
}

```

Add the following to Annex C:

C.2.11 Clause 20: general utilities library [diff.cpp03.utilities]

20.1.5

Change: Containers now access their allocators through the `allocator_traits` template.

Rationale: Simplifies writing new allocators.

Effect on original feature: `allocator_traits` supplies default definitions for many allocator type names and operations. Containers written by users conforming to the original allocator requirements will not necessarily support allocators written to the simpler set of requirements in this standard.

20.7.4

Change: Minimal support for garbage-collected regions

Rationale: Required by new feature.

Effect on original feature: Valid C++ 2003 code, compiled without traceable pointer support, that interacts with newer C++ code using regions declared reachable may have different runtime behavior.

Clause 17: library introduction [diff.cpp14.library]

17.6.3.5

Change: `allocator_traits` supports a single trait for allocator propagation, rather than decomposing into separate traits for copy-assignment, move-assignment, and `swap`.

Rationale: Combinations of inconsistent propagation traits added significant complexity to containers, without demonstrating any real benefit.

Effect on original feature: The two member traits `propagate_on_container_copy_assignment` and `propagate_on_container_move_assignment` are now explicitly coupled to the third member trait, `propagate_on_container_swap`. Any allocator that customized these traits separately will now either propagate in all cases, or never, depending entirely on the value of `propagate_on_container_swap::value`.

Add the following to Annex D:

D.x Deprecated allocator traits bits [depr.alloc.traits]

```
namespace std {
    template <class Alloc> struct allocator_traits {
        typedef propagate_on_container_swap propagate_on_container_copy_assignment;
        typedef propagate_on_container_swap propagate_on_container_move_assignment;
    };
}
```

Acknowledgements

References

- [LWG #2103](#) `std::allocator_traits<std::allocator<T>>::propagate_on_container_move_assignment`
- [LWG #2108](#) No way to identify allocator types that always compare equal
- [LWG #2178](#) Annex C update for C++11 on allocators
- [LWG #2183](#) `regex_match_results` missing allocator-aware ctors
- [LWG #2184](#) `regex_match_results` assigment and allocator propagation
- [LWG #2490](#) `<regex>` needs lots of `noexcept`
- [N4258](#) Cleaning up `noexcept` in the Library, Nicolai Josuttis
- [N4511](#) Adding `[nothrow-]swappable` traits, Daniel Krügler